HANOI UNIVERSITY

FACULTY OF INFORMATION TECHNOLOGY

---------o0o--------

REPORT ASSIGNMENT

Multimodal Product Recognition System

Project Name: VisionFinder

Members: Tran Duc Tri (2301040182), Ha Thai Son (2301040163), Dau Tien Duc (2301040044), Nguyen Dinh Huy (2301040085)

Overview: VisionFinder is an AI-powered system that recognizes products from images and retrieves the most relevant items from a product catalog. Instead of relying on keyword search, users can upload a photo of a product, and the system identifies it, returns its details, and suggests visually similar products. The system combines Computer Vision to process the input image, Multimodal AI to understand both images and text descriptions in a shared representation, and Embeddings to convert products into vectors that can be compared by similarity. This helps users find what they want faster, even when they do not know the product name. VisionFinder Home applies this idea to a home & living store of 500 products (furniture, lighting, rugs, decor, storage, plants and textiles) taken from the Amazon Berkeley Objects dataset.

Overview tóm tắt hệ thống

- system: người dùng upload ảnh một sản phẩm, hệ thống nhận ra sản phẩm đó trong catalog (danh mục sản phẩm của cửa hàng), trả về thông tin chi tiết và gợi ý các sản phẩm trông giống.
- điểm mới: người dùng tìm bằng chính bức ảnh, không cần gõ keyword (từ khóa) mô tả.
- công nghệ : Computer Vision xử lý ảnh đầu vào, Multimodal AI hiểu cả ảnh lẫn chữ trong một shared representation (không gian biểu diễn chung), và Embeddings biến sản phẩm thành vector (dãy số) để so sánh độ giống nhau.

Ví dụ: khách thấy một chiếc đèn bàn trong quán cà phê nhưng không biết gọi là gì, chỉ cần chụp ảnh, crop đúng chiếc đèn và upload là hệ thống tìm ra các sản phẩm tương tự trong shop.

I. Problems and Objectives

Problem:

Online stores and product catalogs usually depend on keyword search, which works only when users know the exact name or category of what they want. In practice, people often see

a product in a photo (on social media, in a shop, or in a friend's home) and cannot describe it in words. Three issues follow from this:

1. **Vocabulary mismatch:** users and sellers describe the same product with different words, so keyword search returns irrelevant results or nothing at all.
2. **Slow manual browsing:** scrolling through a large catalog to find a visually similar item is time-consuming and tiring.
3. **Text-only limitation:** a text search box cannot use the most informative input the user already has, which is the image itself.

Objectives:

The goal is to build a pipeline that takes a product image as input and returns the closest matching products from a catalog. Specifically, the system aims to:

1. Convert product images and user query images into embeddings using a pretrained multimodal model.
2. Retrieve the top-k most similar products using similarity search, and return their details to the user.
3. Reject uncertain queries by applying a similarity threshold, so the system reports "no confident match" instead of a wrong answer.
4. Meet target performance: Top-1 accuracy of at least 55%, Top-5 accuracy of at least 70%, and response time under 0.5 seconds per query on a catalog of 500 products.
5. Deliver a stable end-to-end demo, from image upload to displayed results.

Note: the targets were finalized after the initial experiments. They are measured on 999 real query photos (other camera angles of the catalog products) on a laptop CPU; the achieved results are reported in Section V.

Problem: Cửa hàng online thường chỉ cho **tìm bằng từ khóa**, nên người dùng phải biết đúng tên sản phẩm. Khi chỉ có một bức ảnh mà không biết mô tả bằng lời, họ tìm sai hoặc không ra, và lướt cả catalog thì rất mất thời gian.

Objectives: Xây một **pipeline** nhận ảnh sản phẩm và:

- Tạo **embedding** bằng pretrained multimodal (CLIP)
- Trả về **top-k** sản phẩm giống nhất bằng **similarity search**.
- Dùng **threshold** để trả “no confident match” khi không chắc, thay vì đoán sai.
- Đạt Top-1 ≥ 55%, Top-5 ≥ 70%, thời gian phản hồi < 0.5 giây, trên catalog 500 sản phẩm (đo bằng 999 ảnh truy vấn thật).

II. System Architecture

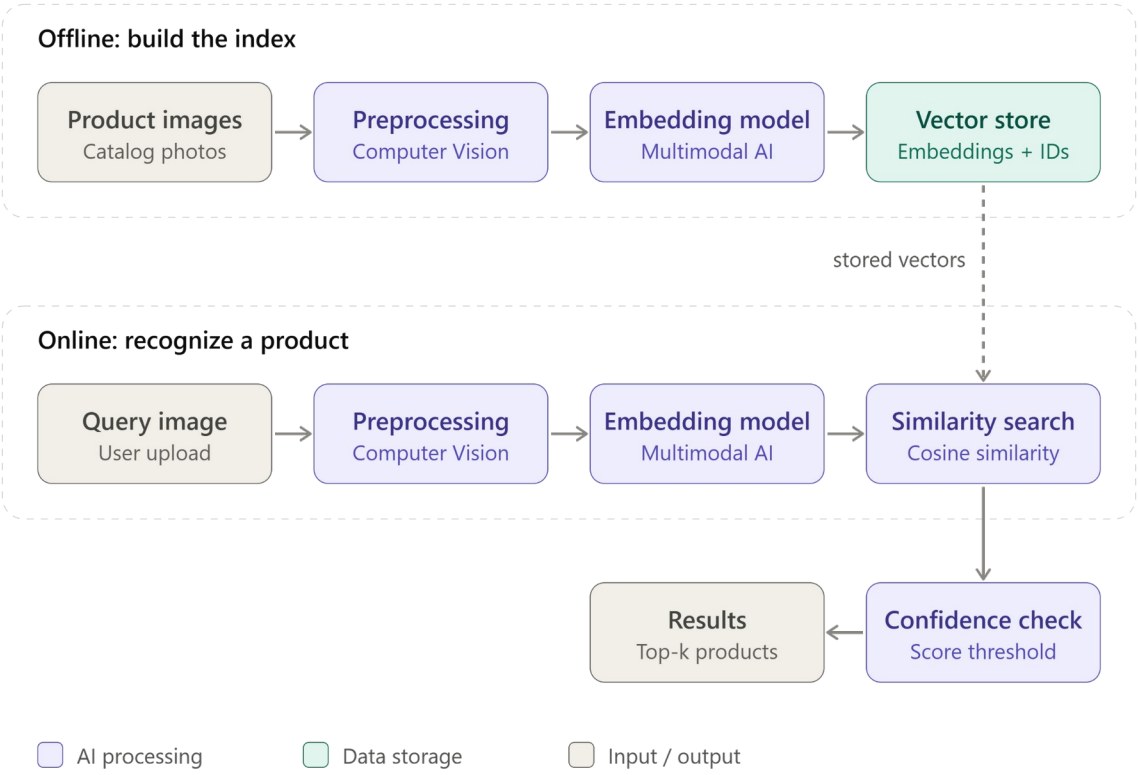


Figure 1. VisionFinder system architecture.

The system has two pipelines. In the offline pipeline, catalog images are preprocessed, converted into embeddings by a multimodal model, and stored in a vector store with their product IDs. In the online pipeline, a user's query image goes through the same preprocessing and embedding steps, is compared with the stored vectors using cosine similarity, and, if the best score passes a confidence threshold, the top-k matching products are returned

Figure 1: Hệ thống có hai luồng. **Offline:** ảnh catalog được xử lý, biến thành vector rồi lưu vào Vector store. **Online:** ảnh người dùng upload đi qua đúng các bước đó, so với vector đã lưu bằng cosine similarity, và nếu điểm vượt ngưỡng thì trả về top-k sản phẩm giống nhất, nếu không thì báo "no confident match".

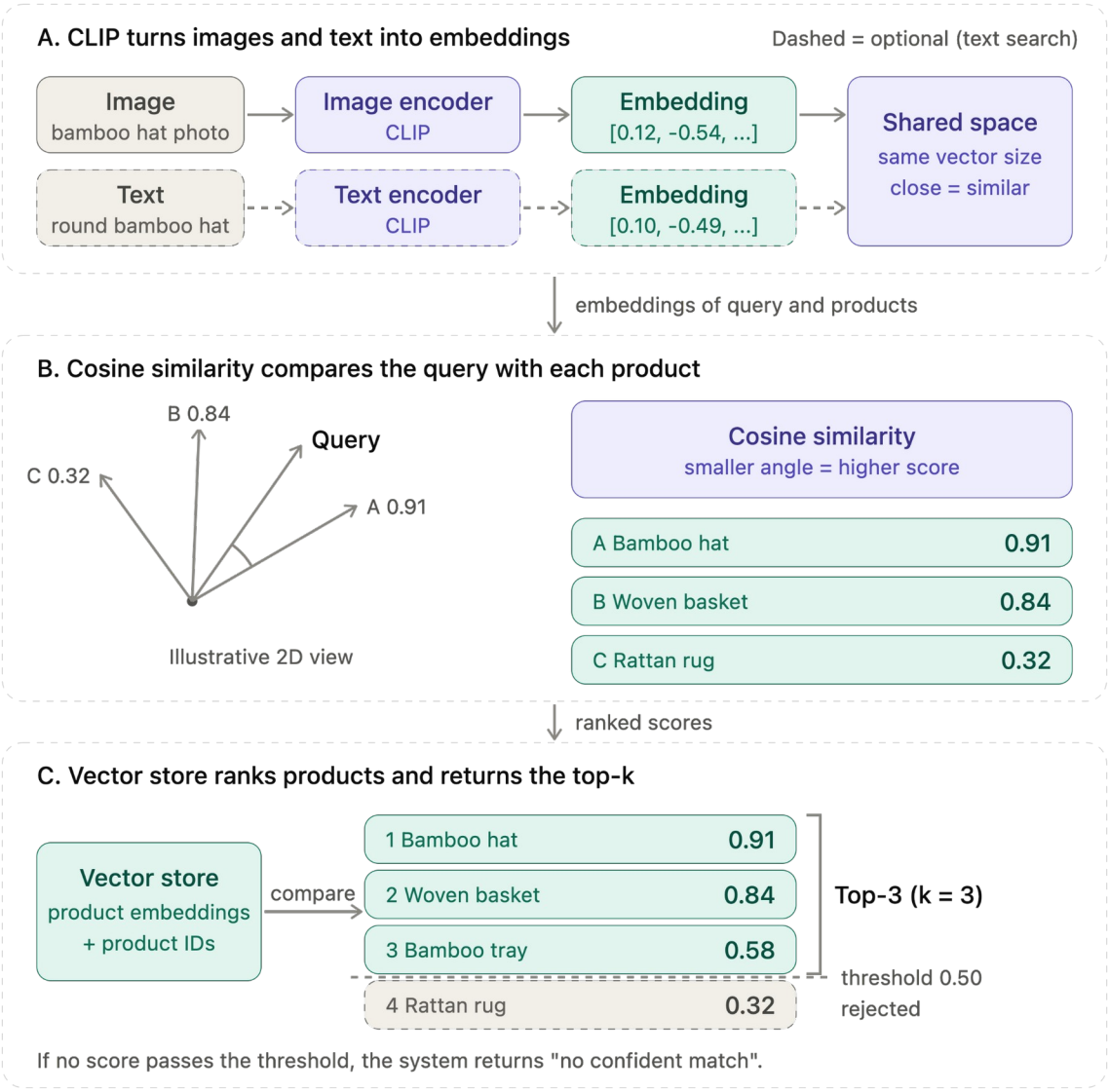


Figure 2. How CLIP embeddings, cosine similarity, the vector store and top-k work together (illustrative values).

CLIP encodes images (and, optionally, text) into embeddings in a shared vector space. The query embedding is compared with each product embedding using cosine similarity, where a smaller angle means a higher score. The vector store ranks the products by score, returns the top-k results, and rejects those below the confidence threshold.

Figure 2 giải thích cách hệ thống biến ảnh thành vector, đo độ giống rồi chọn kết quả, gồm 3 phần đọc từ trên xuống.

A. CLIP tạo embedding: ảnh đi qua **image encoder** của CLIP và thành một **embedding** (vector, ví dụ [0.12, -0.54, ...]). Chữ cũng có thể đi qua **text encoder** để thành vector cùng kích thước. Cả hai nằm trong **shared space**, nơi hai vector càng gần nhau thì nội dung càng giống nhau. Hàng chữ vẽ nét đứt vì nhóm chưa dùng tìm bằng chữ, đó là phần mở rộng.

B. Cosine similarity: hệ thống so vector của ảnh người dùng (**Query**) với vector từng sản phẩm bằng **góc** giữa chúng. Góc càng nhỏ thì điểm càng cao. Ở hình, sản phẩm A (bamboo hat) lệch ít nhất nên được 0.91, còn sản phẩm C (rattan rug) lệch nhiều nên chỉ 0.32.

C. Vector store và top-k: các sản phẩm được xếp theo điểm từ cao xuống thấp. Hệ thống lấy **top-k** (ở đây k = 3) và dùng **threshold** 0.50 để loại những sản phẩm dưới ngưỡng, như rattan rug 0.32. Nếu không sản phẩm nào vượt ngưỡng thì trả về "no confident match".

III. Development Plan

Phase	Task	Assigned	Start Date	End Date	Status
1. Initiation	Repository Setup	Tri	2026-09-14	2026-09-20	Completed
2. Design	Dataset Collection & Image Preprocessing (Computer Vision)	Tri	2026-09-21	2026-10-01	Completed
3. Execution	Embedding Generation with Multimodal Model	Son	2026-10-05	2026-10-11	Completed
3. Execution	Vector Store & Similarity Search	Duc	2026-10-08	2026-10-15	Completed
4. Testing	Confidence Threshold Tuning & Evaluation	Huy	2026-10-16	2026-10-22	Completed
5. Closing	API & Frontend Integration, Testing & Demo	Team	2026-10-23	2026-11-06	Completed

IV. Technical Approaches

Technology: The system uses a Python/FastAPI service for the AI pipeline and a static web frontend (HTML, CSS, JavaScript) for image upload, cropping and result display, connected through a REST API. For Computer Vision, images are preprocessed with Pillow (EXIF rotation, transparent backgrounds flattened to white, padding to a square so the product is not cropped). For Multimodal AI, a pretrained CLIP model (OpenCLIP ViT-B-32, trained on LAION-2B) embeds both images and text descriptions. For Embedding storage and retrieval, vectors are kept in a FAISS IndexFlatIP index; because the vectors are L2-normalized, inner product equals cosine similarity. The catalog contains 500 home & living products from the

Amazon Berkeley Objects dataset (CC BY 4.0), plus 100 holdout products that are not in the catalog and are used for threshold tuning.

Trade-offs: We use a pretrained embedding model with similarity search instead of training a classifier from scratch. The trade-off is lower accuracy on very similar-looking products, but we avoid needing a large labeled dataset and heavy GPU training, and new products can be added by embedding their images without retraining. We also use a similarity threshold: a stricter threshold gives fewer wrong matches but more "no confident match" answers, so it was tuned during evaluation. Finally, we compared two CLIP backbones: ViT-B-16 was 1.6 times slower (158 ms vs 99 ms per query) with almost the same accuracy (59.6% vs 60.0% Top-1, 75.2% vs 74.7% Top-5), so the smaller ViT-B-32 was kept.

V. Progress

Current progress: All planned phases are completed. The FastAPI + CLIP + FAISS backend is connected to the web frontend, which supports photo search (drag-and-drop, paste, crop), text search and "visually similar" suggestions, and the API passes 8/8 automated tests. On 999 query photos (other camera angles of the 500 catalog products), the system reaches 60.0% Top-1 and 74.7% Top-5 accuracy, with an average response time of 99 ms (p95: 107 ms) on a laptop CPU, which meets the targets in Section I. Accuracy is highest for Decor (75.7%) and Storage (72.5%) and lowest for Rugs (44.4%) and Textiles (45.1%), whose extra angles are often close-ups of texture. The confidence threshold was set to 0.825, but it separates catalog and out-of-catalog photos only weakly (27.1% correct accepts vs 18.3% false accepts), mainly because many holdout products are near-identical variants of catalog items. Future work: index several angles per product and tune the threshold with clearly unrelated photos.

VI. AI Disclosure

Human Tasks: [System architecture design, choice of the dataset and product categories, design of the evaluation (multi-view queries and holdout set), and testing and review of all results were carried out by team members.]

AI Tasks: [We used AI assistants to help draft and format this report, to explain technical concepts, and to generate most of the code (data preparation, web frontend, FastAPI/CLIP/FAISS backend, evaluation scripts and tests) from our prompts and feedback. All code was run, tested and reviewed by the team.]